

DL-Based Pothole Detection System

Arpit Kr Singh (CS23I1022), Abdur Rahim (CS23I1019), Shubham Saharan (EC23B1058),

Suvam Sahu (CS23I1038), Vansh Singh (CS23I1028), Ayush Mishra (CS23I1058)

B.Tech / Dual Degree – Computer Science & Engineering

Supervisor: Dr. Suresh Vardhraj

Table of Contents

1. Problem Statement
2. Objectives
3. Assumptions
4. Concept Design
5. Bill of Materials
6. Prototype
7. Outcome & Inferences
8. References

Contents

1	Problem Statement	2
2	Objectives	2
3	Assumptions	2
4	Concept Design	3
4.1	System Architecture	3
4.2	ML Model Design	3
4.3	Mobile App Screens	4
4.4	Data Flow	5
4.5	API Endpoints	6
4.6	Database Schema	6
4.7	Detailed How It Works	6
4.8	Sensor Data Format	7
4.9	ML Model Summary	8
4.10	Tech Stack	8
4.11	Project Timeline	9
5	Bill of Materials	9
6	Prototype	10
6.1	Mobile App	10
6.2	Backend and ML Service	11
6.3	Map Screen	12
6.4	Deployment URLs	13
7	Outcome & Inferences	13
7.1	Model Performance	13
7.2	System Verification	14
7.3	Limitations	14
7.4	Future Work	15
8	References	15

1. Problem Statement

Current navigation systems route vehicles based on travel time or distance. They do not factor in road surface quality. Roads with potholes may appear as the fastest route on a map, but they cause vehicle damage, accidents, and discomfort for passengers.

There is no low-cost, automated system that continuously maps road surface quality using data already available on a smartphone. Manual pothole reporting is slow and unreliable. Municipalities also lack real-time data to prioritize road repairs.

This project builds **RoadSafe**, a system that:

- Uses a smartphone's built-in accelerometer and gyroscope to detect potholes while driving.
- Saves detected pothole locations to a shared cloud database automatically.
- Displays a crowd-sourced pothole map that any user can view.

2. Objectives

1. Collect accelerometer, gyroscope, and GPS data from a smartphone in real time while driving.
2. Train a deep learning model (LSTM with 1D-LBP features) to classify road segments as *pothole* or *smooth road*.
3. Deploy the trained model as a REST API microservice that returns a prediction and confidence score.
4. Build a mobile app (React Native / Expo) that integrates sensor collection, backend communication, and live map display.
5. Persist detected pothole coordinates in a cloud PostgreSQL database and expose them via a RESTful API.
6. Enable any user to view all reported potholes on a live crowd-sourced map from within the app.

3. Assumptions

- The smartphone is mounted in the vehicle (e.g., on a dashboard holder) so that sensor readings faithfully reflect road vibrations.
- GPS is available and has adequate accuracy (± 5 –10 m) for pinning pothole locations on the map.
- The device samples sensors at approximately 50 Hz, which matches the model's sequence length requirement.
- Internet connectivity is available during a trip to send sensor data to the backend in near-real time.

- The Kaggle dataset used for training (pothole sensor data from a controlled driving study) is representative enough for the model to generalize across different vehicles and road types.
- Free-tier cloud deployment (Render) is sufficient for prototype/MVP evaluation. Production scaling is out of scope for this phase.

4. Concept Design

4.1 System Architecture

The system consists of three independently deployed components connected over HTTP, plus a shared PostgreSQL database.

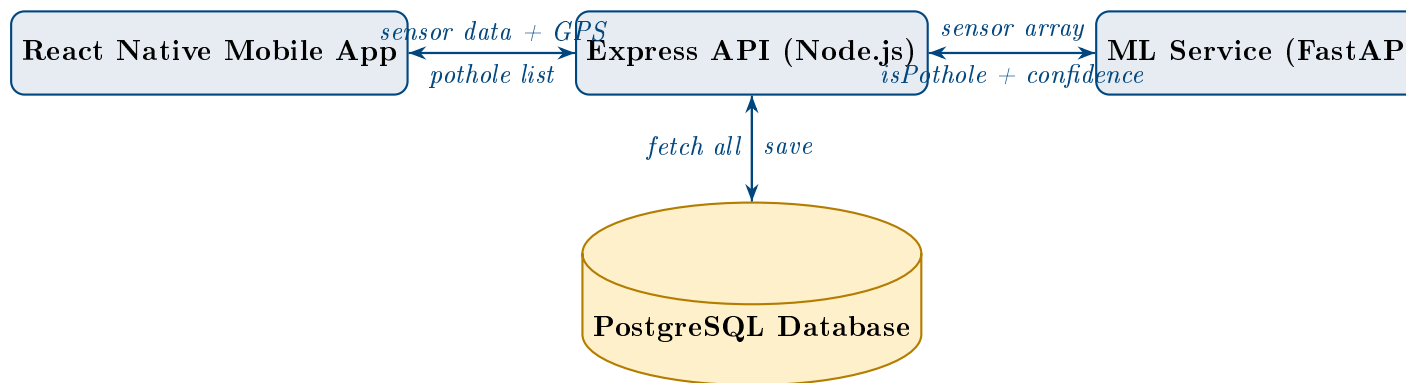


Figure 1: RoadSafe System Architecture

Mobile App Reads sensors at 50 Hz, buffers 50+ readings, and sends them together with GPS coordinates to the Express API.

Express API (Node.js / TypeScript) Validates the request, forwards sensor data to the ML microservice, and saves confirmed potholes to the database. If a new detection falls within 5 m of an existing pothole, the system increases that record's severity score instead of creating a new entry.

ML Microservice (Python / FastAPI) Loads the pre-trained Keras model once at startup. For each incoming request it runs feature engineering and LSTM inference, returning a binary prediction and confidence score.

PostgreSQL Stores pothole records (latitude, longitude, severity, timestamps) managed through Prisma ORM v7.

4.2 ML Model Design

The model takes a sequence of **50 timesteps** $\times$ **68 engineered features** as input and outputs a 2-class softmax probability.

Feature Engineering Pipeline

- **Raw sensors:** accelerometerX/Y/Z, gyroX/Y/Z, speed (7 features)

- **Euclidean magnitudes** of accelerometer and gyroscope vectors (2 features)
- **Derived:** speed-normalized acceleration, impact score, Z-gyro combination (3 features)
- **Rolling standard deviation** over a 50-sample window for accel and gyro (2 features)
- **1D Local Binary Pattern (LBP) histograms:** applied to 7 signals, 8 bins each (56 features)

1D-LBP is adapted from image texture analysis. For each sample in a time series, neighbouring values are compared to the centre value to generate a binary pattern code. A normalized histogram of these codes over a sliding window captures the local vibration texture, which differs distinctly between smooth road segments and pothole impacts.

LSTM Architecture

Table 1: LSTM Model Layer Configuration

Layer	Units	Detail
LSTM 1	128	return_sequences=True, Dropout 0.3
LSTM 2	64	return_sequences=True, Dropout 0.3
LSTM 3	32	return_sequences=False, Dropout 0.3
Dense	64	ReLU
Dense	32	ReLU
Dense	16	ReLU
Output	2	Softmax

Training configuration: Loss: sparse categorical cross-entropy. Optimizer: Adam. Trained for 6 epochs, batch size 64, 80/20 train-test split on the Kaggle pothole sensor dataset.

4.3 Mobile App Screens

The React Native / Expo mobile application provides four main screens:

Dense	64	ReLU
Dense	32	ReLU
Dense	16	ReLU
Output	2	Softmax

Table 1: LSTM Model Layer Configuration

Loss: sparse categorical cross-entropy. Optimizer: Adam. Trained for 6 epochs, batch size 64, 80/20 train-test split.

4.3 Mobile App Screens

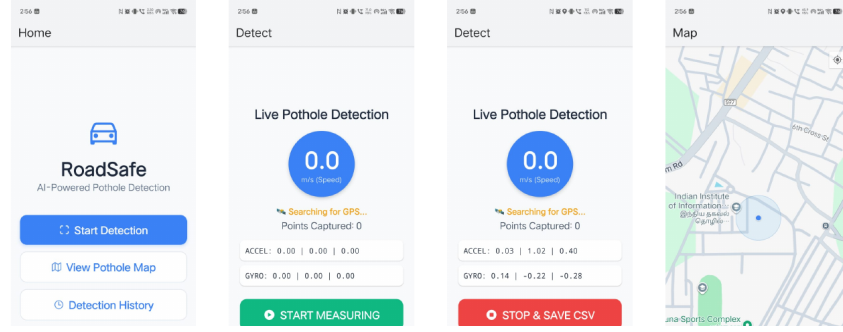


Figure 2: RoadSafe App – Four Main Screens: (a) Home, (b) Detect (idle), (c) Detect (active), (d) Pothole Map

4.4 Data Flow

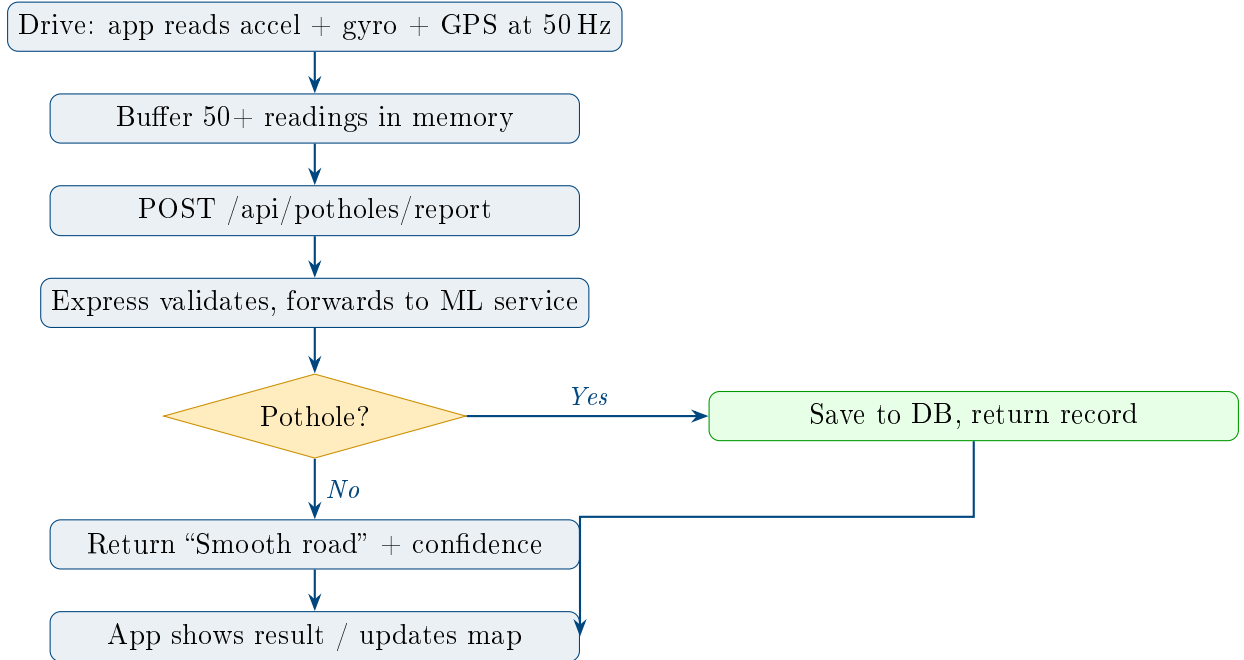


Figure 3: End-to-End Data Flow

4.5 API Endpoints

Table 2: Express API Endpoints

Method	Endpoint	Description
GET	/	Health check
POST	/api/potholes/report	Submit sensor data; ML inference; save if pothole
GET	/api/potholes	Fetch all pothole records for map rendering

The ML microservice exposes a single endpoint:

Table 3: ML Microservice Endpoint

Method	Endpoint	Description
POST	/predict	Accept sensor array; return {isPothole, confidence}

4.6 Database Schema

Table 4: Pothole Table (PostgreSQL via Prisma ORM)

Column	Type	Description
id	String (UUID)	Auto-generated primary key
latitude	Float	GPS latitude of detected pothole
longitude	Float	GPS longitude of detected pothole
severity	Float?	Model confidence score (0–1), nullable
createdAt	DateTime	Auto-set on creation
updatedAt	DateTime	Auto-managed by Prisma ORM

The Prisma schema definition:

```
model Pothole {
  id          String    @id @default(uuid())
  latitude    Float
  longitude   Float
  severity    Float?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}
```

4.7 Detailed How It Works

The full request lifecycle from mobile app to response is described below:

1. The **React Native mobile app** collects real-time sensor data (accelerometerX/Y/Z, gyroscopeX/Y/Z, speed) along with GPS coordinates while the user drives.

2. Once a batch of at least **50 sensor readings** is accumulated, the app sends a **POST** request to the Express API server.
3. The Express server **validates** the incoming data — checking for a minimum of 50 rows and the presence of valid GPS coordinates.
4. The Express server **forwards the raw sensor array** to the Python ML microservice via an internal HTTP POST to `/predict`.
5. The Python service performs **feature engineering** on the raw sensor data:
 - Computes acceleration and gyroscope **magnitudes** (Euclidean norms).
 - Calculates **jerk** (rate of change) for both sensor axes.
 - Computes **rolling standard deviation** over a 50-sample window.
 - Derives **speed-normalized acceleration, impact score, and Z-gyro combo** features.
 - Applies **1D Local Binary Pattern (LBP)** texture analysis to each of the 7 sensor signals and creates 8-bin histogram features (56 features total).
6. The engineered features are **scaled** using a pre-fitted StandardScaler and fed into the Keras LSTM model as a sequence of 50 timesteps $\times$ 68 features.
7. The model returns a **binary classification** (pothole / smooth road) and a **confidence score** (softmax probability).
8. Back in Express, if a pothole is confirmed, it is saved to PostgreSQL with GPS coordinates and the confidence score as severity. Before saving, the backend checks if any existing pothole record is within 5m of the new location. If yes, the severity of that existing record is incremented rather than creating a duplicate entry. This means the more vehicles that detect the same pothole, the higher its severity score gets — and that score is reflected on the map in the app.
9. The result is returned to the mobile app, which **updates the map** with the new pothole marker if detected.

4.8 Sensor Data Format

Each reading in the `sensorData` array must contain the fields described in Table 5. A minimum of **50 readings** is required per request, matching the LSTM model’s sequence length.

Table 5: Sensor Data Fields

Field	Type	Description
accelerometerX	float	Acceleration along X-axis (m/s^2)
accelerometerY	float	Acceleration along Y-axis (m/s^2)
accelerometerZ	float	Acceleration along Z-axis (m/s^2)
gyroX	float	Gyroscope rotation rate along X-axis (rad/s)
gyroY	float	Gyroscope rotation rate along Y-axis (rad/s)
gyroZ	float	Gyroscope rotation rate along Z-axis (rad/s)
speed	float	Device speed (m/s)

4.9 ML Model Summary

Table 6: ML Model Details

Property	Value
Model Type	Deep Learning (Keras .h5)
Architecture	Sequential (LSTM-based)
Input Shape	(50, 68) — 50 timesteps $\times$ 68 features
Classification	Binary (Pothole vs. Smooth Road)
Output	Softmax probabilities $\rightarrow$ argmax for class
Feature Scaling	StandardScaler (pre-fitted, saved as .pkl)
Inference Mode	CPU-only (CUDA_VISIBLE_DEVICES=-1)
Epochs / Batch	6 epochs, batch size 64
Train-Test Split	80/20

4.10 Tech Stack

Table 7: Technology Stack

Layer	Technology
Mobile Client	React Native / Expo
API Gateway	Express v5 (TypeScript / Node.js)
ML Service	FastAPI (Python 3.10+) / Uvicorn
Database	PostgreSQL (cloud-hosted)
ORM	Prisma v7 (with @prisma/adapters-pg)
ML Framework	TensorFlow / Keras
Feature Eng.	pandas, NumPy, scikit-learn, joblib
Package Manager	pnpm (Node.js), pip (Python)
Deployment	Render (free tier)
Version Control	GitHub
Training Data	Kaggle public dataset

4.11 Project Timeline

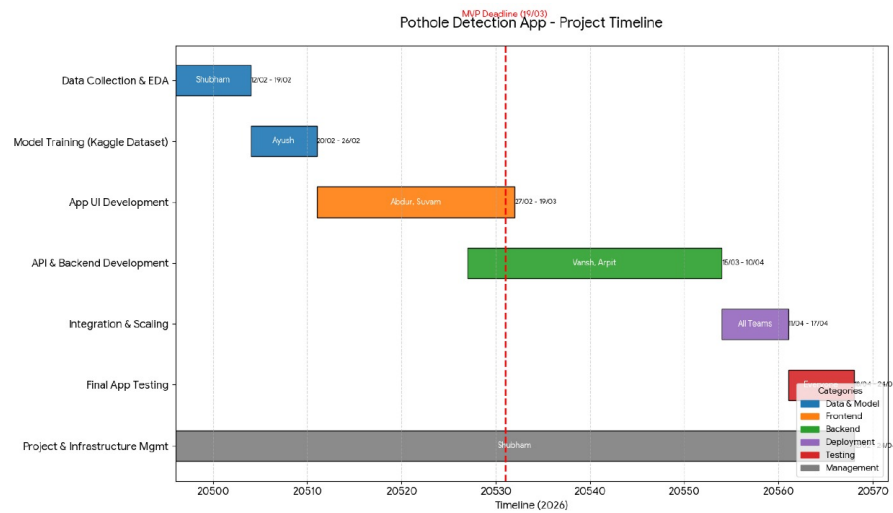


Figure 4: Project Gantt Chart – MVP Deadline: 19 March 2026

Phase	Owner(s)	Period
Data Collection & EDA	Shubham	12/02 – 19/02
Model Training (Kaggle)	Ayush	20/02 – 26/02
App UI Development	Abdur, Suvam	27/02 – 19/03
API & Backend Development	Vansh, Arpit	15/03 – 10/04

Figure 4: Project Gantt Chart – MVP Deadline: 19 March 2026

Table 8: Task Breakdown by Phase

Phase	Owner(s)	Period
Data Collection & EDA	Shubham	12/02 – 19/02
Model Training (Kaggle)	Ayush	20/02 – 26/02
App UI Development	Abdur, Suvam	27/02 – 19/03
API & Backend Development	Vansh, Arpit	15/03 – 10/04
Integration & Scaling	All	11/04 – 17/04
Final App Testing	All	19/04 – 24/04
Project & Infra Mgmt	Shubham	Throughout

5. Bill of Materials

This project uses entirely free and open-source tools. No hardware was purchased beyond the personal smartphones used for testing.

Table 9: Bill of Materials – All Components are Free / Open-Source

Component	Tool / Platform	Cost
Mobile Frontend	React Native, Expo (open source)	Free
API Server	Node.js, Express v5, TypeScript	Free
ML Microservice	Python, FastAPI, Uvicorn	Free
ML Framework	TensorFlow / Keras	Free
Feature Eng.	pandas, NumPy, scikit-learn	Free
Database	PostgreSQL (cloud-hosted)	Free tier
ORM	Prisma v7	Free
Deployment	Render (free tier)	Free
Version Control	GitHub	Free
Training Data	Kaggle (public dataset)	Free
Test Devices	Personal Android smartphones	Owned

6. Prototype

The current prototype is a fully functional MVP with all three layers (mobile, backend, ML service) deployed and communicating in production.

6.1 Mobile App

The app reads live accelerometer and gyroscope data from the device. Figure 5 shows the Detect screen in active mode: sensor values update in real time while a GPS lock is being acquired. Tapping *Stop & Save CSV* ends the session and saves a local CSV file of all sensor readings, accessible from the History screen.

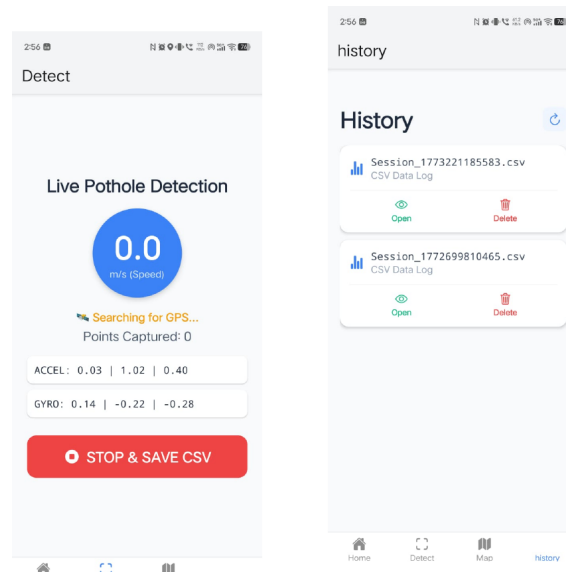


Figure 5: Prototype – Detect Screen (active) and Session History (CSV files)

6.2 Backend and ML Service

Figure 6 shows a test run against the live backend. Two sensor inputs were sent: one for a smooth road and one for a deep pothole. The smooth input came back as “Smooth road detected” with 99.99% confidence. The pothole input saved a record to the database and returned the full entry with GPS coordinates, severity, and timestamp.



Figure 5: Prototype – Detect and History Screens

6.2 Backend and ML Service

Figure 6 shows a test run against the live backend. Two synthetic sensor inputs were sent: one simulating a smooth highway and one simulating a deep pothole. The smooth input returned a “Smooth road detected” response with 99.99% confidence. The pothole input triggered a database save and returned a full pothole record with GPS coordinates, severity, and timestamp.

Figure 6: Backend Test Output – Correct Classification for Both Inputs

Figure 7 shows the JSON response returned by the `GET /api/potholes` endpoint, listing stored pothole records with their coordinates and severity values.

```
Testing: Real Smooth Highway...
Response for Real Smooth Highway: {
  success: true,
  message: 'Smooth road detected. Nothing saved.',
  confidence: 0.9999923706054688
}

Testing: Real Deep Pothole...
Response for Real Deep Pothole: {
  success: true,
  message: 'BOOM! Pothole detected and saved to the map!',
  data: {
    id: 'f55b459f-df0a-489f-b7bb-8ec0d3ecb6d5',
    latitude: 13.0827,
    longitude: 80.2707,
    severity: 1,
    createdAt: '2026-03-11T09:31:38.045Z',
    updatedAt: '2026-03-11T09:31:38.045Z'
  }
}
arpit@arpit-Predator-PHN16-71:~/potholeBackend$
```

Figure 6: Backend test output – correct classification for both inputs

Figure 7 shows the JSON response from GET /api/potholes, listing stored pothole records with their coordinates and severity values.

Figure 7: Pothole Records Returned from the Database API

6.3 Map Screen

The map screen (Figure 8) renders the user's live location near IIITDM Kancheepuram. Detected pothole markers retrieved from the database are overlaid on the map view, enabling crowd-sourced situational awareness for all users of the app.

```
[
  {
    "latitude": 1225.25,
    "longitude": 1.2251,
    "severity": null
  },
  {
    "latitude": 13.0827,
    "longitude": 80.2707,
    "severity": 1
  },
  {
    "latitude": 13.0827,
    "longitude": 80.2707,
    "severity": 1
  },
  {
    "latitude": 13.0827,
    "longitude": 80.2707,
    "severity": 1
  },
  {
    "latitude": 13.0827,
    "longitude": 80.2707,
    "severity": 1
  },
  {
    "latitude": 13.0827,
    "longitude": 80.2707,
    "severity": 1
  }
]
```

Figure 8: Map Screen Showing Live User Location and Detected Pothole Markers

6.4 Deployment URLs

Table 10: Deployed Service URLs

Service	Platform	URL
ML Microservice	Render	https://potholepredictionmodel.onrender.com/predict
Express API	Render	https://potholebackend-ew5n.onrender.com
Database	Cloud	PostgreSQL (cloud-hosted)

7. Outcome & Inferences

7.1 Model Performance

The LSTM model achieved approximately **99% validation accuracy** in just 6 epochs on the Kaggle pothole sensor dataset.

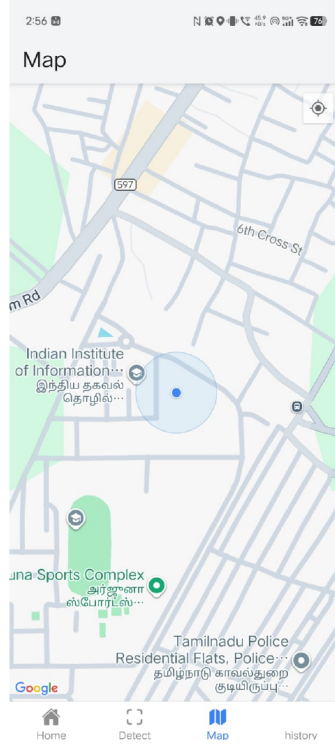


Figure 9: Training and Validation Accuracy / Loss over 6 Epochs

Validation accuracy reaches $\sim 98\%$ by epoch 1 and stays above 99% after that. Validation loss tracks the training loss closely with no sign of overfitting. The 1D-LBP texture features give the model a clear signal to tell pothole impacts apart from normal road vibrations.

7.2 System Verification

All system components were verified end-to-end:

- Smooth road input $\rightarrow$ correctly rejected with 99.99% confidence.
- Pothole input $\rightarrow$ correctly detected, saved to the database, correct response returned.
- Map screen $\rightarrow$ correctly fetches and displays pothole coordinates from the API.
- History screen $\rightarrow$ session CSV files saved and listed correctly.
- When multiple users detect a pothole at the same spot ($\leq 5\text{m}$), the severity score of that entry increases on the map rather than adding a new duplicate pin.

7.3 Limitations

- Training data is from a single controlled study; the model may not generalize well to all road types and vehicles without more diverse data.
- Free-tier Render deployment causes cold-start delays ($\sim 30\text{ s}$) when the service has been idle.

- GPS accuracy ($\pm 5\text{--}10\text{ m}$) means map pins may not align precisely with the actual pothole location.
- Internet is required during detection; offline use is not supported yet.
- The model only classifies pothole vs. smooth; it does not grade severity (minor / moderate / severe).

7.4 Future Work

- **Quality-based routing:** Score routes by pothole density and suggest the smoothest path.
- **Severity classification:** Move from binary to multi-class (minor, moderate, severe).
- **Offline buffering:** Store detections locally when offline and sync when back online.
- **Local data collection:** Collect labelled driving data from Chennai roads to improve accuracy.
- **Municipality dashboard:** Let civic bodies view pothole heat-maps and plan repairs.

8. References

- [1] Allouch, A., Koubaa, A., Abbes, T., & Ammar, A. (2017). RoadSense: Smartphone Application to Estimate Road Conditions Using Accelerometer and Gyroscope. *IEEE Sensors Journal*, 17(14), 4504–4514.
- [2] Hochreiter, S., & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- [3] Ojala, T., Pietikainen, M., & Harwood, D. (1994). Performance evaluation of texture measures with classification based on Kullback discrimination of distributions. In *Proceedings of the 12th IAPR International Conference on Pattern Recognition* (Vol. 1, pp. 582–585).
- [4] Maeda, H., Sekimoto, Y., Seto, T., Kashiya, T., & Omata, H. (2018). Road Damage Detection and Classification Using Deep Neural Networks with Smartphone Images. *Computer-Aided Civil and Infrastructure Engineering*, 33(12), 1127–1141.
- [5] Sattar, S., Li, S., & Chapman, M. (2020). Road Surface Monitoring Using Smartphone Sensors: A Review. *Sensors*, 20(22), 6642.
- [6] Sensor-based Pothole Detection – PMC Reference. *Sensors*, 20, 5564. <https://pmc.ncbi.nlm.nih.gov/articles/PMC7583950>
- [7] dexterdoes. (2024). Pothole Sensor Data [Dataset]. Kaggle. <https://www.kaggle.com/datasets/dexterdoes/pothole-sensor-data>
- [8] FastAPI Documentation. <https://fastapi.tiangolo.com>

- [9] Prisma ORM Documentation. <https://www.prisma.io/docs>
- [10] Expo / React Native Documentation. <https://docs.expo.dev>